

Handout # 1

Date: February 02, 2026

Name: M. WAHAJRoll No. BITP24M059

Code Segment # 1.1

Assumptions (if any): $n = \text{currSize}$

Line#	Code	Frequency of Execution
1	void UnsortedList::display () {	—
2	for (int i=0; i<currSize; i++)	$n+1$
3	cout << arr[i] << " ";	n
4	cout << endl;	1
5	}	—
Total Number of Steps $\rightarrow T(n) =$		$2n+2$
Big Oh		$O(n)$

Code Segment # 1.2

Assumptions (if any): $n = \text{currSize}$

Line#	Code	Frequency of Execution
1	void UnsortedList::displayAlternate () {	—
2	for (int i=0; i<currSize; i = i+2)	$(n/2) + 1$
3	cout << arr[i] << " ";	$n/2$
4	cout << endl;	1
5	}	
Total Number of Steps $\rightarrow T(n) =$		$n+2$
Big Oh		$O(n)$

Handout # 1

Date: February 02, 2026

Code Segment # 1.3

Assumptions (if any): $n = \text{currSize}$

Line#	Code	Worst Case Frequency of Execution	Best Case Frequency of Execution
1	bool UnsortedList::remove1 (int index, int& val) {	—	—
2	if (index >= 0 && index < currSize) {	1	1
3	val = arr[index];	1	0
4	arr[index] = arr[currSize-1];	1	0
5	currSize--;	1	0
6	return true;	1	0
7	}	—	—
8	else	0	0
9	return false;	0	1
10	}	—	—
Total Number of Steps $\rightarrow T(n) =$		5	2 constant
Big Oh		$O(1)$	$O(1)$

Handout # 2

Date: February 02, 2026

Name: Muhammad HakeemRoll No: B-1214024

Code Segment # 2.1

Assumptions (if any): n = currSize

Line#	Code	Frequency of Execution	Frequency of Execution
1	bool UnsortedList::remove2 (int index, int& val) {	—	—
2	if (index >= 0 && index < currSize) {	1	1
3	val = arr[index];	1	0
4	for (int i=index+1; i<currSize; i++) {	n	0
5	arr[i-1] = arr[i];	n-1	0
6	}	—	—
7	currSize--;	1	0
8	return true;	→ 1	0
9	}	—	—
10	else	—	—
11	return false;	0	1
12	}	—	—
Total Number of Steps → $T(n) =$		$2n+3$	2
Big Oh		$O(n)$	$O(1)$

Code Segment # 2.2

Assumptions (if any): _____

Line#	Code	best case Frequency of Execution	Worst case Frequency of Execution	
1	bool linearSearch (int* a, int n, int key) {	—	—	
2	bool found = false;	1	1	1
3	for (int i=0; i < n && !found; i++) {	2	n+1	n+1
4	if (a[i] == key)	1	n	n
5	found = true;	1	1	0
6	}	—	—	—
7	return found;	1	1	1
8	}	—	—	—
Total Number of Steps $\rightarrow T(n) =$		6	2n+3	2n+4
Big Oh		$O(1)$	$O(n)$	$O(n)$

Best case:- when the key is found at the index 0.

Worst case:- when value is not found at last element or index

3rd column:- when value is not found.